Algorítmica

Tema 2. Algoritmos «divide y vencerás»

Francisco Javier Cabrerizo Lorite

Curso académico 2024-2025

ETS de Ingenierías Informática y de Telecomunicación. Universidad de Granada

Índice

1. Introducción

2. Determinación del umbral

3. Análisis

4. Ejemplos de aplicación

Objetivos

- Saber identificar soluciones distintas para un problema y evaluar su bondad.
- Ser consciente de la importancia del análisis de la eficiencia de un algoritmo como paso previo a su implementación.
- Entender la técnica de resolución de un problema por división en problemas de menor tamaño.
- Conocer y saber aplicar los esquemas básicos de la técnica de diseño de algoritmos «divide y vencerás».
- Conocer los criterios de aplicación de la técnica de diseño de algoritmos «divide y vencerás».

Introducción

Introducción

¿Cuándo usar la técnica «divide y vencerás»?

- La instancia del problema se puede descomponer en subinstancias de menor tamaño del mismo problema.
- Las subinstancias se pueden resolver independientemente unas de otras.
- Las subinstancias son disjuntas, sin solapamiento.
- La solución de la instancia original del problema puede obtenerse mediante la combinación de las soluciones de las subinstancias.

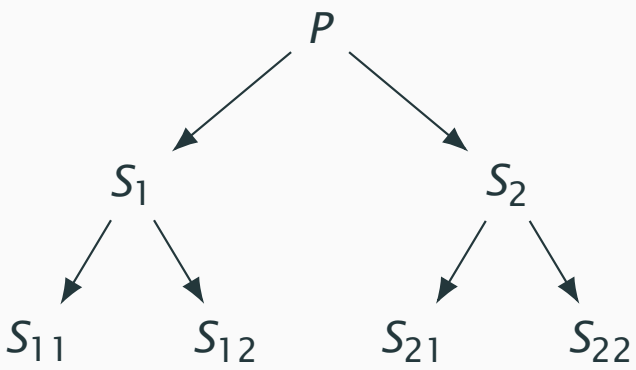
Etapas de la técnica «divide y vencerás»

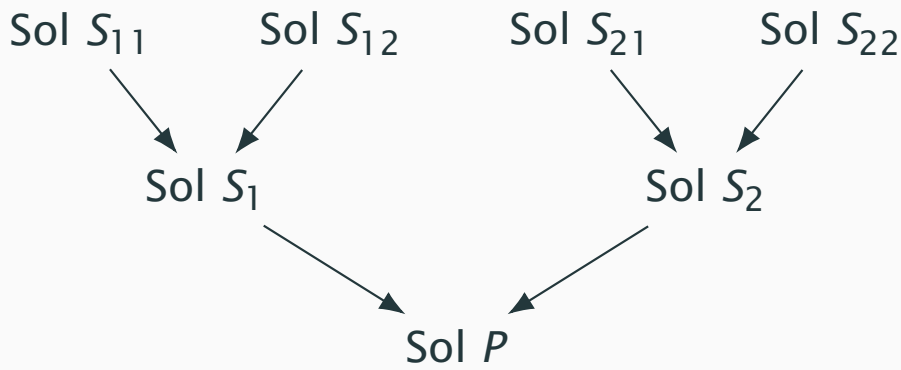
- **Dividir.** Consiste en descomponer la instancia original de tamaño n del problema en k subinstancias ($1 \leq k \leq n$) del mismo problema pero de menor tamaño (cada una con una entrada de tamaño n_k, donde $0 < n_k < n$).

- **Conquistar.** Consiste en «vencer» sucesiva e independientemente las subinstancias, bien directamente si su tamaño es suficientemente pequeño (casos «base») o bien de forma recursiva.

- **Combinar.** Consiste en unir las soluciones de las subinstancias para obtener la solución de la instancia original.

Introducción





- Si las subinstancias son todavía relativamente grandes se aplicará de nuevo esta técnica hasta alcanzar subinstancias lo suficientemente pequeñas como para ser resueltas directamente.

- Esto sugiere el uso de la recursividad en las implementaciones de estos algoritmos (soluciones eficientes).

- Las ecuaciones de recurrencias son naturales en esta técnica de diseño de algoritmos.

Esquema general de la técnica «divide y vencerás»

```
función divide_y_vencerás(P, n)
    si P es suficientemente pequeño entonces
        Solución ← base(P, n)
    si no
        Dividir P en k subinstancias S_1, S_2, ..., S_k, de menor tamaño n_1, n_2, ..., n_k
        para todo i = 1, ..., k hacer
            Solución_i ← divide_y_vencerás(S_i, n_i)
        fin para todo
        Solución ← combinar(Solución_1, ..., Solución_k)
    fin si
    devolver Solución
fin función
```

Consideraciones de la técnica «divide y vencerás»

- El número de subinstancias (k) suele ser pequeño e independiente de la instancia concreta del problema que se a resolver.
- Si $k = 1$, se habla de reducción o simplificación.
- `base(P,n)` hace referencia a un caso base o a un algoritmo capaz de resolver directamente el problema (sin más recursividad):
  - Debe ser eficiente para instancias de tamaño pequeño.
  - Su rendimiento en instancias de tamaño grande es indiferente.

Problema: elevar un número real a un entero

Sean r y n un número real y un número entero, respectivamente. Se requiere un algoritmo que realice la operación r^n.

Primera solución: algoritmo base

- Multiplicar r por sí mismo un total de n veces:

$$r^n = r \cdot r \cdots r \qquad (n \text{ veces})$$

```
función elevar(r, n)
    resultado ← 1.0
    para todo i = 1, . . . , n hacer
        resultado ← resultado ∗ r
    fin para todo
    devolver resultado
fin función
```

- $T(n) \in \Theta(n)$

Segunda solución: algoritmo «divide y vencerás»

- ¿Se puede dividir el problema?
  - Sí: $r^n = r \cdot r^{n-1}$
  - Esta división no es muy interesante.
- ¿Alguna otra división?
  - $r^n = r^{n/2} \cdot r^{n/2}$
  - Esta división es más eficiente.
- Expresión:

$$r^n = \begin{cases} r^{n/2} \cdot r^{n/2} & \text{si } n \text{ es par} \\ r \cdot r^{n/2} \cdot r^{n/2} & \text{si } n \text{ es impar} \end{cases}$$

Segunda solución: algoritmo «divide y vencerás»

- La instancia del problema se puede dividir en dos subinstancias de menor tamaño ($n/2$).
- Las subinstancias se pueden resolver independientemente (sacando partido de las propiedades de las potencias).
- La combinación es la multiplicación del resultado de calcular $r^{n/2}$ por el resultado de calcular $r^{n/2}$.
- Además del método básico mostrado anteriormente, existen los casos base $r^0 = 1$ y $r^1 = r$:

$$r^n = \begin{cases} 1 & \text{si } n = 0 \\ r & \text{si } n = 1 \\ r^{n/2} \cdot r^{n/2} & \text{si } n > 1 \text{ y } n \text{ es par} \\ r \cdot r^{n/2} \cdot r^{n/2} & \text{si } n > 1 \text{ y } n \text{ es impar} \end{cases}$$

Segunda solución: algoritmo «divide y vencerás»

- **Dividir.** La instancia original del problema r^n se divide en subinstancias de tamaño $n/2$. En realidad, en una única subinstancia (simplificación).

- **Conquistar.** La subinstancia se va resolviendo de forma recursiva hasta llegar al caso base, el cual se resuelve de la siguiente forma:

$$\begin{cases} 1 & \text{si } n = 0 \\ r & \text{si } n = 1 \end{cases}$$

- **Combinar.** Sea S_1 la solución de la subinstancia, la solución S de la instancia original se calcula como:

$$S = \begin{cases} S_1 \cdot S_1 & \text{si } n \text{ es par} \\ r \cdot S_1 \cdot S_1 & \text{si } n \text{ es impar} \end{cases}$$

Segunda solución: algoritmo «divide y vencerás»

```
función elevarDV(r, n)
    si n = 0 entonces
        devolver 1
    si no si n = 1 entonces
        devolver r
    si no
        resultado ← elevarDV(r, n/2)
        si n % 2 = 0 entonces
            devolver resultado * resultado
        si no
            devolver r * resultado * resultado
        fin si
    fin si
fin función
```

- Orden de complejidad:

$$T(n) = T(n/2) + \Theta(1) \in \Theta(\log(n))$$

Determinación del umbral

Determinación del umbral

Cuestiones clave de la técnica «divide y vencerás»

- ¿Cómo descomponer la instancia del problema en subinstancias?
- ¿Cómo resolver las subinstancias?
- ¿Cómo combinar las soluciones?
- ¿Merece la pena hacer esto?

Determinación del umbral

- Supongamos una instancia de un problema P, de tamaño n, que sabemos que puede resolverse con un algoritmo A cuyo tiempo de ejecución es:

$$T_A(n) \leq cn^2$$

- Se divide P en 3 subinstancias de tamaño $n/2$, siendo cada una de ellas del mismo tipo que P, y consumiendo un tiempo lineal la combinación de sus soluciones: $T_{\text{com}}(n) \leq dn$.

- Se obtiene así un nuevo algoritmo B, «divide y vencerás», cuyo tiempo de ejecución es:

$$T_B(n) = 3T_A(n/2) + T_{\text{com}}(n) \leq 3T_A(n/2) + dn \leq \frac{3}{4}cn^2 + dn$$

- El algoritmo B tiene un tiempo de ejecución mejor que el algoritmo A, ya que disminuye la constante oculta.

Determinación del umbral

- Si cada subinstancia se resuelve de nuevo con el algoritmo «divide y vencerás» podemos hacer un tercer algoritmo C, recursivo, cuyo tiempo de ejecución sería:

$$T_C(n) = \begin{cases} T_A(n) & \text{si } n \leq n_0 \\ 3\,T_C(n/2) + T_{\text{com}}(n) & \text{si } n > n_0 \end{cases}$$

- El algoritmo C es mejor en eficiencia que los algoritmos A y B:

$$T_C(n) \leq bn^{1.58}$$

- Al valor n_0 se le denomina umbral y es fundamental para que funcione bien la técnica de diseño de algoritmos «divide y vencerás».

Determinación del umbral

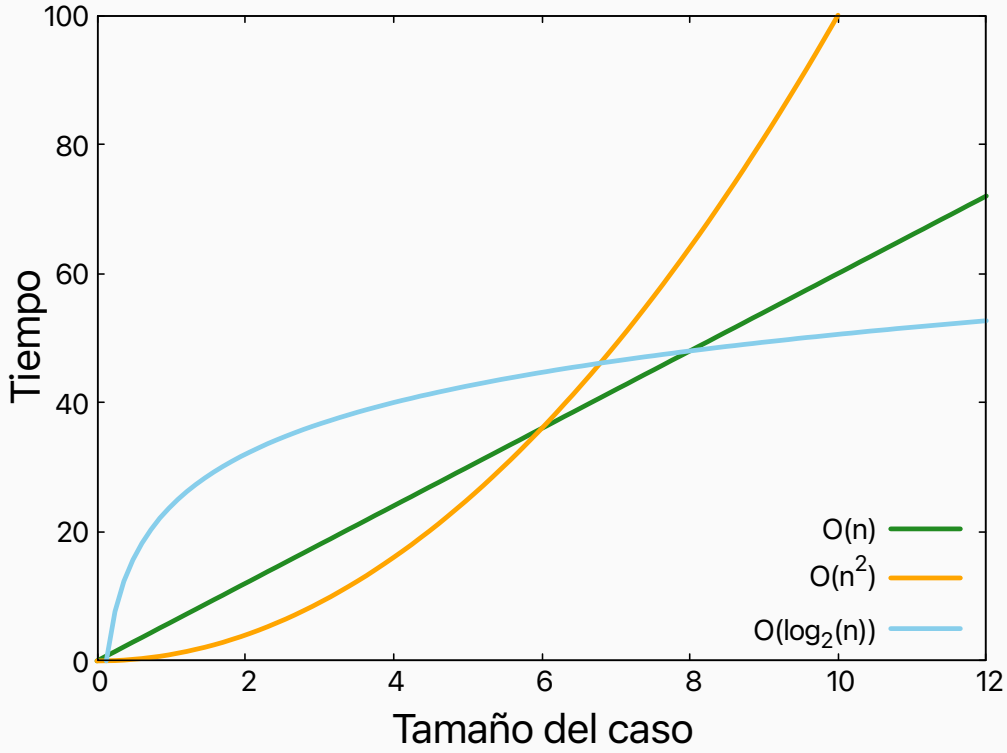
Tamaño muy pequeño de la instancia

- Muchas llamadas recursivas.
- Ineficiente debido a tantas llamadas recursivas.

Alternativa

- ¿Se posee un algoritmo que resuelva el problema de forma directa?
- Ejecutarlo cuando P sea suficientemente pequeño.
- ¿Cuándo es suficientemente pequeño? Se comparan los tiempos de ejecución del algoritmo diseñado mediante la técnica «divide y vencerás» con los del algoritmo que resuelve el problema de forma directa.

Determinación del umbral



$$\text{Mejor} = \begin{cases} \mathrm{O}(n^2) & \text{si } n \leq 6 \\ \mathrm{O}(n) & \text{si } 6 < n \leq 8 \\ \mathrm{O}(\log_2(n)) & \text{si } n > 8 \end{cases}$$

Determinación del umbral

Mergesort e inserción

- El algoritmo base para el problema de ordenación será el algoritmo de ordenación por inserción ($\mathrm{O}(n^2)$).

- Para determinar cuándo el tamaño es suficientemente pequeño como para dejar de aplicar mergesort (algoritmo «divide y vencerás») calculamos empíricamente el tiempo de ejecución de los dos algoritmos:
  - Se mide el tiempo de ejecución de ambos algoritmos para distintos tamaños del problema.
  - Se calcula la constante oculta de inserción: $T_{\text{inserción}}(n) \leq c_1\, n^2$
  - Se calcula la constante oculta de mergesort: $T_{\text{mergesort}}(n) \leq c_2\, n \log_2(n)$
  - Se calcula el valor de n a partir de la igualdad: $c_1\, n^2 = c_2\, n \log_2(n)$

Determinación del umbral

Inserción

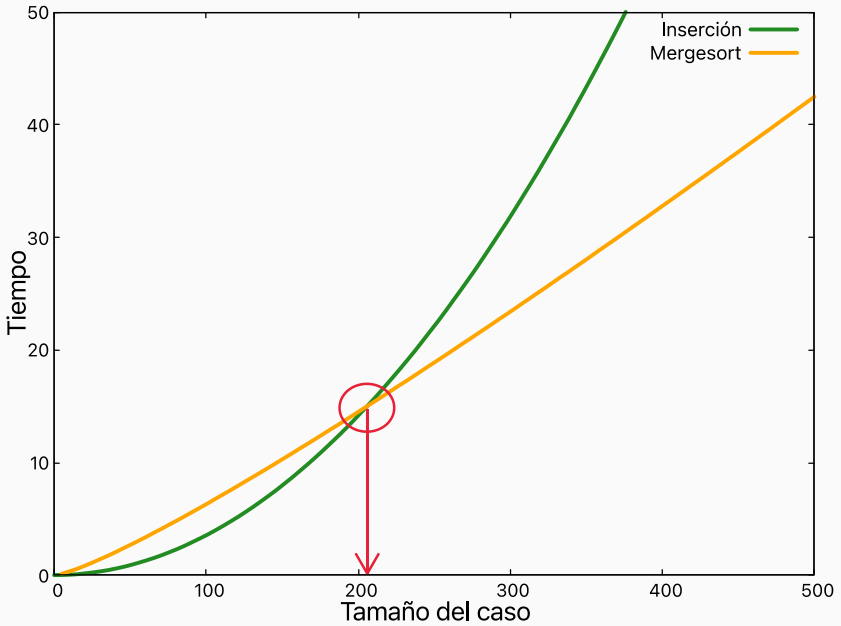
n	$T(n)$
10 000	45 425
20 000	140 155
30 000	303 081
40 000	542 304
50 000	856 232
60 000	1 228 797
70 000	1 673 289
80 000	2 198 850
90 000	2 787 441
100 000	3 449 736

$c_1 = 3.54 \cdot 10^{-4}$

Mergesort

n	$T(n)$
10 000	2 267
20 000	2 599
30 000	3 992
40 000	5 461
50 000	6 968
60 000	8 413
70 000	9 932
80 000	11 629
90 000	11 835
100 000	12 038

$c_2 = 9.48 \cdot 10^{-3}$



¿En qué punto comienza a ser mergesort mejor que inserción?

$$c_1 n^2 = c_2 n \log_2(n)$$

n se puede calcular gráficamente o de forma aproximada si no se dispone de métodos matemáticos para ello

Determinación del umbral

Mergesort e inserción

- Pseudocódigo del algoritmo mergesort tras resolver el problema de la determinación del umbral:

```
procedimiento mergesort(V[], inicio, fin)
    si fin − inicio ≤ 208 entonces
        devolver inserción(V[], inicio, fin)
    si no
        medio ← (inicio + fin) / 2
        mergesort(V[], inicio, medio)
        mergesort(V[], medio + 1, fin)
        combinar(V[], inicio,  medio, fin)
fin si
fin procedimiento
```

Análisis

Análisis

- Si un algoritmo contiene una llamada a sí mismo, su tiempo de ejecución puede describirse por una recurrencia que da el tiempo de ejecución para una instancia de tamaño n en función de subinstancias de menor tamaño.

- Los algoritmos diseñados mediante la técnica de diseño «divide y vencerás» presentan ecuaciones de recurrencias del tipo:

$$T(n) = \begin{cases} d(n) & \text{si } n \leq n_0 \\ kT(n/b) + g(n) & \text{si } n > n_0 \end{cases}$$

- k representa el número de subinstancias y n/b el tamaño de estas.

- $g(n)$ denota el coste de descomponer la instancia del problema inicial en las k subinstancias y el de combinar las soluciones para obtener la solución de la instancia original.

- $d(n)$ representa el coste de resolver el problema de forma directa (caso base o algoritmo no recursivo).

Fórmula maestra

$$T(n) = kT(n/b) + g(n)$$

- Si $g(n) \in \Theta(n^a)$, $a \in \mathbb{R}^{\geq 0}$, el orden de complejidad de la solución de la ecuación de recurrencias anterior es:

$$\begin{aligned} \Theta(n^a) &\quad \text{si } k < b^a \\ \Theta(n^a \log(n)) &\quad \text{si } k = b^a \\ \Theta(n^{\log_b(k)}) &\quad \text{si } k > b^a \end{aligned}$$

- El orden de complejidad depende de la relación entre el número de subinstancias (k), el tamaño de estas (n/b) y la dificultad de dividir y combinar (a).

La importancia de las condiciones (1/2)

- El número de subinstancias en que se divide la instancia original del problema tiene influencia en el orden de complejidad:

$$T(n) = 2T(n/2) + c \quad \Rightarrow \quad T(n) \in \mathrm{O}(n^{\log_2(2)}) = \mathrm{O}(n)$$
$$T(n) = 4T(n/2) + c \quad \Rightarrow \quad T(n) \in \mathrm{O}(n^{\log_2(4)}) = \mathrm{O}(n^2)$$
$$T(n) = 8T(n/2) + c \quad \Rightarrow \quad T(n) \in \mathrm{O}(n^{\log_2(8)}) = \mathrm{O}(n^3)$$

- El coste de la descomposición en subinstancias y de la combinación de soluciones tiene influencia en el orden de complejidad:

$$T(n) = 2T(n/2) + c \quad \Rightarrow \quad T(n) \in \mathrm{O}(n^{\log_2(2)}) = \mathrm{O}(n)$$
$$T(n) = 2T(n/2) + n \quad \Rightarrow \quad T(n) \in \mathrm{O}(n \log(n))$$
$$T(n) = 2T(n/2) + n^2 \quad \Rightarrow \quad T(n) \in \mathrm{O}(n^2)$$

La importancia de las condiciones (2/2)

- Que las subinstancias en que se divide la instancia original del problema sean de tamaño similar tiene influencia en el orden de complejidad:

$$T(n) = 2T(n/2) + n \quad \Rightarrow \quad T(n) \in \mathrm{O}(n \log(n))$$
$$T(n) = T(1) + T(n-1) + n \quad \Rightarrow \quad T(n) \in \mathrm{O}(n^2)$$

- El tamaño de las subinstancias en que se divide la instancia original del problema tiene influencia en el orden de complejidad:

$$T(n) = 2T(n/4) + c \quad \Rightarrow \quad T(n) \in \mathrm{O}(n^{\log_4(2)}) = \mathrm{O}(n^{0.5}) = \mathrm{O}(\sqrt{n})$$
$$T(n) = 2T(n/2) + c \quad \Rightarrow \quad T(n) \in \mathrm{O}(n^{\log_2(2)}) = \mathrm{O}(n)$$
$$T(n) = 2T(n-1) + c \quad \Rightarrow \quad T(n) \in \mathrm{O}(n^2)$$

Condiciones para que «divide y vencerás» sea ventajoso

- Selección cuidadosa de cuándo usar el algoritmo que resuelve el problema de forma directa (determinación del umbral de recursividad).
- Poder descomponer eficientemente la instancia original del problema en sub-instancias y combinar de forma eficiente a partir de las soluciones parciales.
- El número de subinstancias debe ser razonablemente pequeño.
- Las subinstancias deben tener aproximadamente el mismo tamaño.
- Las subinstancias deben ser del menor tamaño posible.

Permite diseñar algoritmos asintóticamente eficientes

Ejemplos de aplicación

Búsqueda binaria

Enunciado

Sea $V[1..n]$ un vector ordenado por orden no decreciente, $V[i] \leq V[j]$ para $1 \leq i \leq j \leq n$, y sea x un elemento a buscar. Formalmente, se quiere encontrar el índice i tal que $1 \leq i \leq n+1$ y $V[i-1] < x \leq V[i]$.

1	2	3	4	5
3	7	25	41	53

Si $x = 25$ $\Rightarrow$ $i = 3$

Si $x = 15$ $\Rightarrow$ $i = 3$

Si $x = 67$ $\Rightarrow$ $i = 6$

Si $x = 2$ $\Rightarrow$ $i = 1$

Primera solución: búsqueda secuencial

- La forma simple y directa de resolver el problema consiste en buscar hasta llegar al final o hasta encontrar un elemento que no sea menor que x.

```
función búsqueda_secuencial(V[], n, x)
    i ← 1
    mientras i ≤ n  &  x > V[i] hacer
        i ←  i + 1
    fin mientras
    devolver i
fin función
```

- Orden de complejidad:

$$T(n) \in \mathrm{O}(n)$$

Búsqueda binaria

Segunda solución: búsqueda binaria

- Para acelerar la búsqueda, se puede buscar el elemento x o bien en la primera mitad del vector o bien en la segunda.
- Para determinar en qué mitad buscar se compara x con el elemento del vector que ocupa la posición $k = n/2$.
- Si $x \leq V[k]$ se puede restringir la búsqueda a $V[1..k]$. En otro caso, se busca en $V[k + 1..n]$.
- Es un caso de reducción o simplificación: la solución de toda instancia se reduce a una única instancia más pequeña (concretamente de tamaño mitad).

Segunda solución: búsqueda binaria

```
función búsqueda_recursiva(V[], inicio, fin, x)
    si inicio = fin entonces
        devolver inicio
    si no
        centro ← (inicio + fin) / 2
        si x ≤ V[centro] entonces
            devolver búsqueda_recursiva(V[], inicio, centro, x)
        si no
            devolver búsqueda_recursiva(V[], centro + 1, fin, x)
        fin si
    fin si
fin función
función búsqueda_binaria(V[], n, x)
    si x > V[n] entonces
        devolver n + 1
    si no
        devolver búsqueda_recursiva(V[], 1, n, x)
    fin si
fin función
```

- Orden de complejidad:

$$T(n) = T(n/2) + \Theta(1) \in \Theta(\log(n))$$

Ordenación rápida (quicksort)

Enunciado

Sea $V[1..n]$ un vector de n elementos. El problema consiste en ordenar estos elementos por orden ascendente, es decir, $V[i] \leq V[j]$ para $1 \leq i \leq j \leq n$.

Quicksort

- Algoritmo basado en la técnica «divide y vencerás» propuesto por C.A.R. Hoare en 1962.
- Es el algoritmo de ordenación general más eficiente (en caso promedio).
- Idea básica:
  - Determinar un elemento (pivote) que divida al vector en dos partes: una que contenga los elementos menores que él y otra con los mayores o iguales.
  - Ordenar los dos subvectores obtenidos.
  - Combinar los dos subvectores ordenados para obtener el vector inicial ordenado.

Quicksort

- **Dividir.** Se divide el vector $V[1..n]$ en dos subvectores independientes que se pueden ordenar por separado (mejor si son de tamaño parecido). Para ello, se selecciona un elemento (pivote) de modo que los elementos menores que él queden a la izquierda del vector, y los mayores o iguales a la derecha.

- **Conquistar.** Las subinstancias se van resolviendo de forma recursiva hasta llegar al caso base:
  - Al llegar al caso base de un elemento ($n = 1$), el vector estará ordenado.
  - La penúltima llamada recursiva (vector de $n = 2$ elementos) también hará que el vector resultante esté ordenado, ya que se ha pivotado y el elemento mayor estará en la segunda posición, mientras que el menor estará en la primera.
  - En las llamadas recursivas anteriores, pasará lo mismo debido a que se va pivotando cada vez que se ejecuta la función.

- **Combinar.** Debido al uso del pivote y gracias a las llamadas recursivas no se requiere la combinación de soluciones.

Quicksort

```
procedimiento quicksort(V[], inicio, fin)
    si inicio < fin entonces
        pos_pivote ← pivotar(V[], inicio, fin)
        quicksort(V[], inicio, pos_pivote −1)
        quicksort(V[], pos_pivote + 1,  fin)
    fin si
fin procedimiento
```

- Orden de complejidad. Depende de la posición en la que termine el pivote elegido:
  - El pivote divide al vector en dos subvectores de igual tamaño (mejor caso).
  - El pivote deja un subvector con 1 elemento y otro subvector con $n - 1$ elementos (peor caso).

Quicksort: elección del pivote (1/2)

- Cualquiera puede diseñar su propio algoritmo quicksort: la elección del pivote condiciona el tiempo de ejecución.
- El pivote puede ser cualquier elemento en el dominio (no es necesario que esté en el vector):
  - Podría ser la media de los elementos seleccionados del vector.
  - Podría elegirse aleatoriamente.
- Pivotes usuales:
  - La mediana de un mínimo de tres elementos del vector.
  - El elemento medio del vector.

Quicksort: elección del pivote (2/2)

- El empleo de la mediana de tres elementos no tiene justificación teórica.
- Si se quiere usar el concepto de mediana, se debería elegir como pivote la mediana del vector porque lo divide en dos subvectores de igual tamaño:
  - Escoger el $(n/2)^{\text{o}}$ mayor elemento.
  - Elegir tres elementos al azar y coger su mediana (suele reducir el tiempo de ejecución en un 5%).
- Una elección más rápida consiste en elegir el mayor de los dos primeros elementos del vector.

Ordenación rápida (quicksort)

5	89	35	14	24	15	37	13	20	7	70

5	89	35	14	24	15	37	13	20	7	70

7	14	5	13	15	35	37	89	20	24	70

¿Cómo realizar eficientemente la partición?

Quicksort: partición (1/2)

- Es fácil diseñar un algoritmo de partición con un orden de complejidad lineal.
- Es importante que la constante oculta sea lo más pequeña posible para que quicksort sea competitivo.
- Se puede recorrer el vector una sola vez, pero empezando por los dos extremos.

Quicksort: Partición (2/2)

- Sea $V[k..l]$ el vector.
- Sea $p = V[k]$ el pivote (el primer elemento).
- Los punteros i y j se inician en k y l, respectivamente.
- El puntero i se incrementa hasta que $V[i] \geq p$ y el puntero j se decrementa hasta que $V[j] < p$.
- Entonces, se intercambian $V[i]$ y $V[j]$. Este proceso se repite mientras $i < j$.
- Finalmente, $V[k]$ y $V[j]$ se intercambian para poner el pivote en la posición correcta.

Quicksort

- La implementación de la función `pivotar` del algoritmo quicksort podría ser:

```
función pivotar(V[], inicio, fin)
    pivote ← V[inicio]
    i ← inicio + 1
    j ← fin
    mientras i ≤ j hacer
        mientras i ≤ j & V[i] < pivote hacer
            i ← i + 1
        fin mientras
        mientras i ≤ j & V[j] ≥ pivote hacer
            j ← j − 1
        fin mientras
        si i < j entonces
            intercambiar(V[i], V[j])
        fin si
    fin mientras
    si inicio < j entonces
        intercambiar(V[inicio], V[j])
    fin si
    devolver j
fin función
```

Quicksort

- **Orden de complejidad en el peor caso.** El pivote es el primer (último) elemento del vector:

$$T(n) = T(1) + T(n-1) + \Theta(n) \in \mathrm{O}(n^2)$$

  Tan malo como el peor caso de los métodos de ordenación por inserción y selección.

- **Orden de complejidad en el mejor caso.** El pivote es la mediana:

$$T(n) = 2T(n/2) + \Theta(n) \in \Omega(n \log(n))$$

- **Orden de complejidad en caso promedio.** Se ha demostrado que es:

$$T(n) \in \mathrm{O}(n \log(n))$$

  En promedio, se comporta mejor que los demás algoritmos de ordenación.

Selección

Enunciado

Sea $V[1..n]$ un vector de n elementos. Se desea conocer qué elemento se situaría en la i-ésima posición del vector en caso de que este estuviese ordenado.

Ejemplo de uso

- Cálculo de la mediana.

Primera solución: algoritmo base

- Ordenar el vector por orden creciente y devolver el elemento que hubiese en la i-ésima posición.

- El orden de complejidad depende del algoritmo de ordenación empleado (es la operación más costosa que se realiza).

Selección

Idea general

- No es necesario que el vector esté totalmente ordenado.
- Lo único necesario es que el elemento *i*-ésimo esté ordenado.

¿Podemos usar algo ya conocido?

- La función pivotar de quicksort.
- Al terminar su ejecución, en la *j*-ésima posición del vector se encuentra el pivote y todos los elementos a su izquierda son menores que él, por lo que se encuentra ordenado en su posición.

Segunda solución: algoritmo «divide y vencerás»

- **Dividir.** Se colocan los elementos menores que un pivote a la izquierda del vector, y los mayores o iguales a la derecha. La posición del pivote, `posPivote`, estará ordenada. Entonces:
  - Si la posición *i* buscada es la posición del pivote, `posPivote`, hemos terminado.
  - Si la posición *i* buscada es inferior a `posPivote`, se divide la instancia original en una subinstancia (vector *V* desde la primera posición hasta `posPivote` -1).
  - Si la posición *i* buscada es superior a `posPivote`, se divide la instancia original en una subinstancia (vector *V* desde `posPivote` $+1$ hasta el último elemento).
- **Conquistar.** Se vuelve a calcular el pivote para el subvector generado, aplicando recursivamente este proceso hasta que se encuentre la *i*-ésima posición o se llegue al caso base (vector de un elemento).
- **Combinar.** Como la función de pivotar deja en `posPivote` el elemento que se está buscando, y solo se divide la instancia del problema en una subinstancia, no es necesario realizar combinación ni operaciones adicionales.

Segunda solución: algoritmo «divide y vencerás»

```
función selección(V[], inicio, fin, i)
    si inicio = fin entonces
        devolver V[inicio]
    si no
        posPivote ← pivotar(V[], inicio, fin)
        si posPivote = i entonces
            devolver V[posPivote]
        si no si i < posPivote entonces
            devolver selección(V[], inicio, posPivote −1, i)
        si no
            devolver selección(V[], posPivote + 1, fin, i)
        fin si
    fin si
fin función
```

Segunda solución: algoritmo «divide y vencerás»

- **Orden de complejidad en el peor caso.** El vector se divide en dos subvectores de tamaño 1 y $n-1$, respctivamente:

$$T(n) = T(1) + T(n-1) + \Theta(n) \in \mathrm{O}(n^2)$$

- **Orden de complejidad en el mejor caso.** El vector se divide en dos subvectores de igual tamaño ($n/2$):

$$T(n) = T(n/2) + \Theta(n) \in \Omega(n)$$

- **Orden de complejidad en caso promedio.** En promedio, se tiene que:

$$T(n) \in \mathrm{O}(n)$$

  Es muy improbable que se dé el peor caso.

Multiplicación de enteros muy grandes

Enunciado

Sean dos números enteros positivos A y B con n dígitos cada uno. Se desea calcular la multiplicación $C = AB$.

Primera solución: algoritmo base

- Multiplicación clásica:

$$\begin{array}{r} 1\,2\,3\,4 \\ \times\, 5\,6\,7\,8 \\ \hline 9\,8\,7\,2 \\ 8\,6\,3\,8 \\ 7\,4\,0\,4 \\ 6\,1\,7\,0 \\ \hline 7\,0\,0\,6\,6\,5\,2 \end{array}$$

- Orden de complejidad:

$$T(n) \in \Theta(n^2)$$

Multiplicación de enteros muy grandes

Segunda solución: algoritmo «divide y vencerás» (primera aproximación)

- Se debe poder obtener la solución de la instancia original del problema en base a subinstancias de menor tamaño.

$$1\,234 = 12 \times 100 + 34$$

$$5\,678 = 56 \times 100 + 78$$

$$\begin{aligned}(12 \times 100 + 34)(56 \times 100 + 78) = \\ = 12 \times 56 \times 10\,000 + \\ + 100(12 \times 78 + 34 \times 56) + \\ + (34 \times 78) \\ = 7\,006\,652\end{aligned}$$

- Se reduce la multiplicación de cuatro cifras a 4 multiplicaciones de dos cifras, más 3 sumas y varios desplazamientos.

Segunda solución: algoritmo «divide y vencerás» (primera aproximación)

- El procedimiento general consiste en dividir los dos números, A y B, de n cifras, en dos partes, A_i y A_d, y B_i y B_d, de $n/2$ cifras cada una:

$$A = A_i 10^{n/2} + A_d$$
$$B = B_i 10^{n/2} + B_d$$

- El producto AB se calcula entonces como sigue:

$$\begin{aligned} AB &= (A_i 10^{n/2} + A_d)(B_i 10^{n/2} + B_d) \\ &= A_i B_i 10^n + (A_i B_d + A_d B_i) 10^{n/2} + A_d B_d \end{aligned}$$

- Las multiplicaciones $A_i B_i$, $A_i B_d$, $A_d B_i$ y $A_d B_d$ se realizan usando este procedimiento recursivamente.

- Se reduce la multiplicación de dos números de n cifras a 4 multiplicaciones de números de $n/2$ cifras, 2 desplazamientos (multiplicaciones por potencias de 10) y 3 sumas.

Multiplicación de enteros muy grandes

Segunda solución: algoritmo «divide y vencerás» (primera aproximación)

```
función mult_enteros_grandes_DV(A, B, n)
    si n es pequeño entonces
        devolver AB
    si no
        Obtener A_i, A_d, B_i, B
        M_1 ← mult_enteros_grandes_DV(A_i, B_i, n/2)
        M_2 ← mult_enteros_grandes_DV(A_i, B_d, n/2)
        M_3 ← mult_enteros_grandes_DV(A_d, B_i, n/2)
        M_4 ← mult_enteros_grandes_DV(A_d, B_d, n/2)
        aux ← sumar(M_2, M_3)
        M_1 ← desplazar_derecha(M_1, n)
        aux ← desplazar_derecha(aux, n/2)
        C ← sumar(M_1, aux, M_4)
        devolver C
    fin si
fin función
```

- Orden de complejidad:

$$T(n) = 4T(n/2) + \Theta(n) \in \Theta(n^2) \Rightarrow \text{Reducir el número de multiplicaciones (4)}$$

Tercera solución: algoritmo de Karatsuba

- Propuesto por Anatolii Alexeevitch Karatsuba en 1960.
- Sean:

$$M_1 = (A_i + A_d)(B_i + B_d) = (A_i B_i) + (A_i B_d + A_d B_i) + A_d B_d$$
$$M_2 = A_i B_i$$
$$M_3 = A_d B_d$$

- Por tanto:

$$A_i B_d + A_d B_i = M_1 - M_2 - M_3$$

- Ahora se puede calcular:

$$AB = M_2 10^n + (M_1 - M_2 - M_3)10^{n/2} + M_3$$

- 1 multiplicación de tamaño $n \Rightarrow$ 3 multiplicaciones de tamaño $n/2$.

Tercera solución: algoritmo de Karatsuba

```
función karatsuba(A, B, n)
    si n es pequeño entonces
        devolver AB
    si no
        Obtener A_i, A_d, B_i, B_d
        S_1 ← sumar(A_i, A_d)
        S_2 ← sumar(B_i, B_d)
        M_1 ← karatsuba(S_1, S_2, n/2)
        M_2 ← karatsuba(A_i, B_i, n/2)
        M_3 ← karatsuba(A_d, B_d, n/2)
        aux ← sumar(M_1, −M_2, −M_3)
        M_2 ← desplazar_derecha(M_2, n)
        aux ← desplazar_derecha(aux, n/2)
        C ← sumar(M_2, aux, M_3)
        devolver C
    fin si
fin función
```

Tercera solución: algoritmo de Karatsuba

- La división de los números A y B en A_i, A_d, B_i y B_d, es del orden de $\Theta(1)$.
- Se crean tres subinstancias de tamaño $n/2$ para resolver el problema.
- Los desplazamientos (multiplicaciones por potencias de 10), las sumas y sustracciones se asumen que son del orden de $\Theta(n)$.
- Por tanto, el orden de complejidad del algoritmo de Karatsuba es:

$$T(n) = 3T(n/2) + \Theta(n) \in \Theta(n^{\log_2(3)}) = \Theta(n^{1.585})$$

Multiplicación de matrices

Enunciado

Sean dos matrices cuadradas A y B de n filas y n columnas ($n \times n$). Se desea calcular la multiplicación $C = AB$.

Primera solución: algoritmo base

- Multiplicación clásica de matrices:

$$A = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \dots & a_{nn} \end{bmatrix} \qquad B = \begin{bmatrix} b_{11} & b_{12} & \dots & b_{1n} \\ b_{21} & b_{22} & \dots & b_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ b_{n1} & b_{n2} & \dots & b_{nn} \end{bmatrix}$$

$$C = \begin{bmatrix} c_{11} & c_{12} & \dots & c_{1n} \\ c_{21} & c_{22} & \dots & c_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ c_{n1} & c_{n2} & \dots & c_{nn} \end{bmatrix} \qquad c_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}$$

Multiplicación de matrices

Primera solución: algoritmo base

- La implementación del método clásico de multiplicación de matrices requiere calcular n^2 entradas:

```
procedimiento multiplicar_matrices(A, B, C, n)
    para todo i = 1, . . . , n hacer
        para todo j = 1, . . . , n hacer
            C[i][j] ← 0
            para todo k = 1, . . . , n hacer
                C[i][j] = C[i][j] + A[i][k] ∗ B[k][j]
            fin para todo
        fin para todo
    fin para todo
fin procedimiento
```

- Orden de complejidad:

$$T(n) \in \Theta(n^3)$$

Segunda solución: algoritmo «divide y vencerás» (primera aproximación)

- Descomponer cada matriz en submatrices que puedan multiplicarse entre sí, reduciendo el tamaño de cada matriz.

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \quad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} \quad C = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$$

- Las submatrices C_{11}, C_{12}, C_{21} y C_{22}, se pueden calcular como sigue:

$$\begin{aligned} C_{11} &= A_{11}B_{11} + A_{12}B_{21} \\ C_{12} &= A_{11}B_{12} + A_{12}B_{22} \\ C_{21} &= A_{21}B_{11} + A_{22}B_{21} \\ C_{22} &= A_{21}B_{12} + A_{22}B_{22} \end{aligned}$$

- Se necesitan 8 multiplicaciones de matrices de tamaño $n/2$ y 4 sumas de matrices cuyo orden de complejidad es $\Theta(n^2)$:

$$T(n) = 8T(n/2) + 4n^2 \in \Theta(n^3)$$

Tercera solución: algoritmo de Strassen

- Volker Strassen descubrió en 1969 una forma de multiplicar las matrices que reducía el número de multiplicaciones totales (muchas sumas y restas).

- La idea básica es que las matrices se pueden multiplicar empleando menos multiplicaciones que las 8 requeridas aparentemente en la definición.

$$
\begin{aligned}
M_1 &= (A_{11} + A_{22})(B_{11} + B_{22}) \\
M_2 &= (A_{21} + A_{22})B_{11} \\
M_3 &= A_{11}(B_{12} - B_{22}) \\
M_4 &= A_{22}(B_{21} - B_{11}) \\
M_5 &= (A_{11} + A_{12})B_{22} \\
M_6 &= (A_{21} - A_{11})(B_{11} + B_{12}) \\
M_7 &= (A_{12} - A_{22})(B_{21} + B_{22})
\end{aligned}
$$

$$
C = \begin{bmatrix} M_1 + M_4 - M_5 + M_7 & M_3 + M_5 \\ M_2 + M_4 & M_1 - M_2 + M_3 + M_6 \end{bmatrix}
$$

Tercera solución: algoritmo de Strassen

- **Dividir.** El problema de la multiplicación de dos matrices cuadradas de tamaño $n \times n$ se divide en 7 subinstancias M_1, M_2, M_3, M_4, M_5, M_6, y M_7, de tamaño $n/2 \times n/2$.

- **Conquistar.** Las 7 subinstancias M_1, M_2, M_3, M_4, M_5, M_6, y M_7, son independientes y se resuelven por separado, bien de forma recursiva o bien directamente (caso base).

- **Combinar.** Las soluciones de las 7 subinstancias se combinan de la siguiente forma para obtener la solución de la instancia original del problema:

$$C_{11} = M_1 + M_4 - M_5 + M_7$$
$$C_{12} = M_3 + M_5$$
$$C_{21} = M_2 + M_4$$
$$C_{21} = M_1 - M_2 + M_3 + M_6$$

Tercera solución: algoritmo de Strassen

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \quad B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} \quad C = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$$

$$M_1 = (A_{11} + A_{22})(B_{11} + B_{22}) = (1 + 4)(5 + 8) = 65$$
$$M_2 = (A_{21} + A_{22})B_{11} = (3 + 4)5 = 35$$
$$M_3 = A_{11}(B_{12} - B_{22}) = 1(6 - 8) = -2$$
$$M_4 = A_{22}(B_{21} - B_{11}) = 4(7 - 5) = 8$$
$$M_5 = (A_{11} + A_{12})B_{22} = (1 + 2)8 = 24$$
$$M_6 = (A_{21} - A_{11})(B_{11} + B_{12}) = (3 - 1)(5 + 6) = 22$$
$$M_7 = (A_{12} - A_{22})(B_{21} + B_{22}) = (2 - 4)(7 + 8) = -30$$

$$C_{11} = M_1 + M_4 - M_5 + M_7 = 65 + 8 - 24 + (-30) = 19$$
$$C_{12} = M_3 + M_5 = (-2) + 24 = 22$$
$$C_{21} = M_2 + M_4 = 35 + 8 = 43$$
$$C_{22} = M_1 - M_2 + M_3 + M_6 = 65 - 35 + (-2) + 22 = 50$$

$$C = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

Tercera solución: algoritmo de Strassen

```
función strassen(A, B, n)
    si n es suficientemente pequeño entonces
        devolver AB
    si no
        Obtener A11, A12, A21, A22, y B11, B12, B21, B22
        M1 ← strassen(A11 + A22, B11 + B22, n/2)
        M2 ← strassen(A21 + A22, B11, n/2)
        M3 ← strassen(A11, B12 − B22, n/2)
        M4 ← strassen(A22, B21 − B11, n/2)
        M5 ← strassen(A11 + A12, B22, n/2)
        M6 ← strassen(A21 − A12, B11 + B22, n/2)
        M7 ← strassen(A12 − A22, B21 + B22, n/2)
        C11 ← M1 + M4 − M5 + M7
        C12 ← M3 + M5
        C21 ← M2 + M4
        C22 ← M1 − M2 + M3 + M6
        C ← combinar(C11,C12,C21,C22)
    fin si
    devolver C
fin función
```

Tercera solución: algoritmo de Strassen

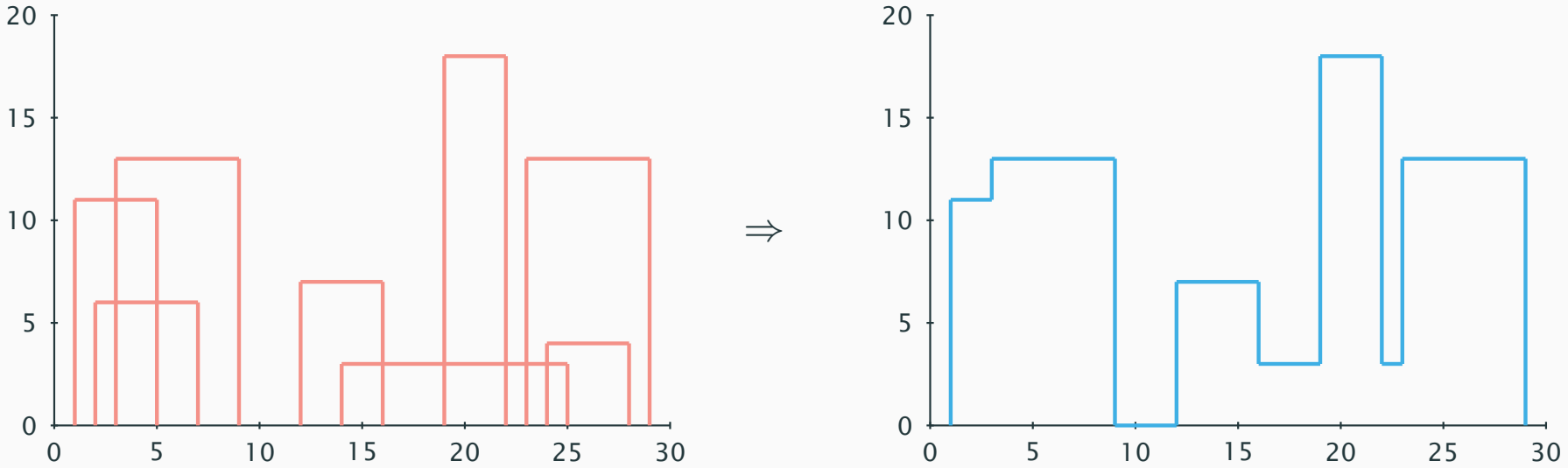
- Las matrices A y B deben ser cuadradas y su tamaño tiene que ser potencia de 2.
- Si no se cumple, se añaden y rellenan filas y columnas con el valor 0 hasta que la matriz tenga el tamaño requerido.
- Se crean 7 subinstancias de tamaño $n/2$ para resolver el problema.
- Hay k sumas y restas que se asume que son del orden de $\Theta(n^2)$.
- Por tanto, el orden de complejidad del algoritmo de Strassen es:

$$T(n) = 7T(n/2) + \Theta(n^2) \in \Theta(n^{\log_2(7)}) = \Theta(n^{2.807})$$

La línea del horizonte

Enunciado

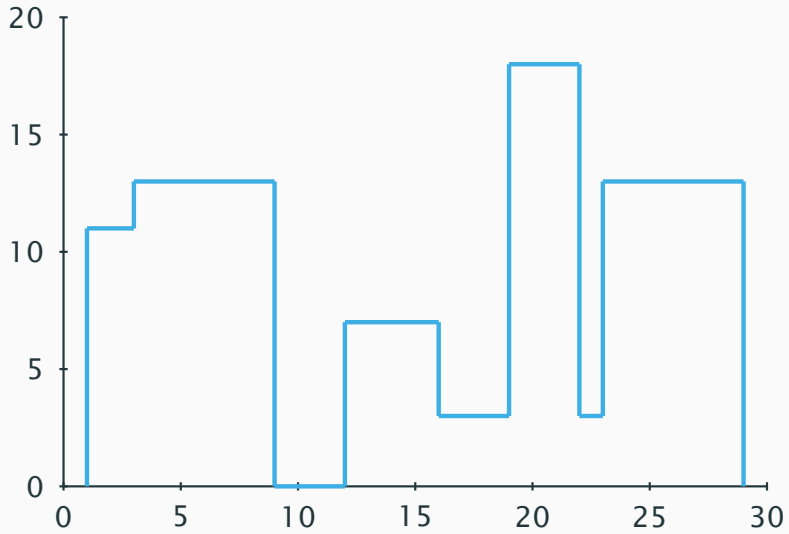
Se dispone de un conjunto de n edificios. Cada edificio E_i, $i \leq n$, se representa como una tripleta (l_i, h_i, r_i) en la que l_i y r_i representan la coordenada x izquierda y derecha del edificio y h_i su altura. Se desea obtener la línea del horizonte (lista de coordenadas x y alturas conectando los edificios de izquierda a derecha).



La línea del horizonte

Entrada {(3, 13, 9), (1, 11, 5), (12, 7, 16), (14, 3, 25), (19, 18, 22), (2, 6, 7), (23, 13, 29), (24, 4, 28)}

Salida {(1,11), (3,13), (9,0), (12,7), (16,3), (19,18), (22,3), (23,13), (29,0)}



Algoritmo «divide y vencerás»

- **Dividir.** Se divide la instancia original del problema en 2 subinstancias de tamaño lo más parecido posible ($n/2$), conteniendo los edificios de la parte izquierda y de la parte derecha, respectivamente.

  La división se puede conseguir en un orden de complejidad $\mathrm{O}(n)$ si se asume que los edificios están ordenados por componente l.

- **Conquistar.** Las dos subinstancias se resuelven de forma recursiva hasta llegar al caso base (el conjunto de edificios contiene un único edificio). En tal caso, el resultado es:

$$\{(l_1, h_1), (r_1, 0)\}$$

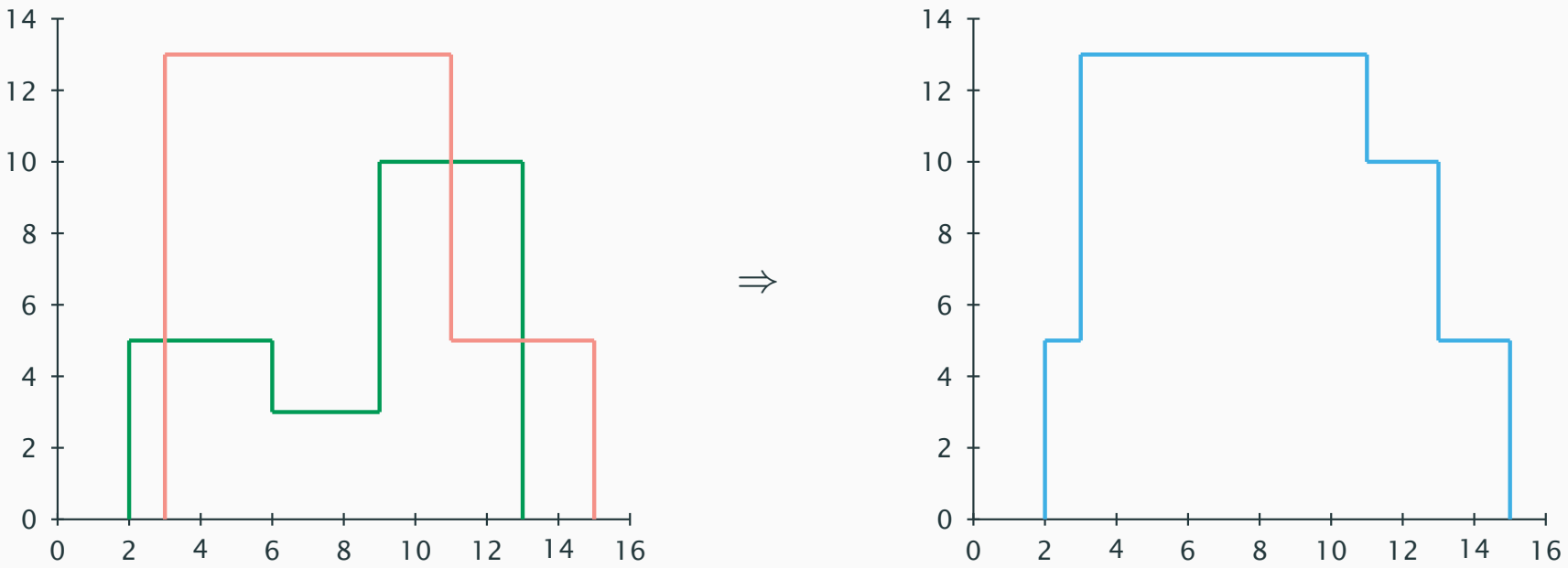
Algoritmo «divide y vencerás»

- **Combinar.** La combinación de las soluciones de las dos subinstancias se realiza de la siguiente forma:
  - Se inicia la altura actual de la línea del horizonte a 0 ($h_{\texttt{skyline}} = 0$).
  - Se comparan los puntos de ambas soluciones empezando por el extremo izquierdo y se escoge el que tenga la coordenada menor.
  - Si la altura del punto elegido es mayor que $h_{\texttt{skyline}}$, se añade la coordenada y la altura del punto elegido a la solución y se actualiza $h_{\texttt{skyline}}$ con la altura del punto elegido.
  - Si la altura del punto elegido es menor que $h_{\texttt{skyline}}$, este se actualiza con la altura del punto elegido o con la altura del último punto elegido de la otra solución, la que sea mayor. Se añade la coordenada y el valor de $h_{\texttt{skyline}}$ a la solución.
  - Se elige el siguiente punto con la menor coordenada y se repite el proceso.
  - Una vez que se han escogido todos los puntos de una de las soluciones, se añaden los de la otra sin necesidad de realizar ningún cálculo adicional.
  - Para evitar añadir dos puntos consecutivos con igual altura, se compara con la altura de último punto añadido y, en caso de ser igual, no se añade.

La línea del horizonte

Algoritmos «divide y vencerás»

- $S_1 = \{(2,5), (6,3), (9,10), (13,0)\}$
- $S_2 = \{(3,13), (11,5), (15,0)\}$
- Combinación de S_1 y $S_2 = \{(2,5), (3,13), (11,10), (13,5), (15,0)\}$



La línea del horizonte

Algoritmo «divide y vencerás»

```
función skyline(E, n)
    si n = 1 entonces
        devolver {(l1,h1),(r1, 0)}
    si no
        Dividir E en dos subconjuntos E1 y E2, con los edificios de E1 teniendo su componente l menor que la de los edificios de E2
        S1 ← skyline(E1, n/2)
        S2 ← skyline(E2, n/2)
        S ← combinar(S1, S2)
    fin si
    devolver S
fin función
```

- Orden de complejidad:

$$T(n) = 2\,T(n/2) + \Theta(n) \in \Theta(n \log(n))$$

Bibliografía

Bibliografía

Aclaración

- El contenido de las diapositivas es esquemático y representa un apoyo para las clases teóricas.

- Se recomienda completar los contenidos del tema 2 con apuntes propios tomados en clase y con la bibliografía principal de la asignatura.

Por ejemplo


G. Brassard and P. Bratley.
***Fundamentals of Algorithmics*.**
Prentice Hall, Englewwod Cliffs, New Jersey, 1996.


T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein.
***Introduction to Algorithms*.**
The MIT Press, Cambridge, MA, 2022.


J. L. Verdegay.
***Lecciones de Algorítmica*.**
Editorial Técnica AVICAM, 2017.